

Clase 4: Flujos de Implementación, Entorno Algorítmico y Validación

Fase Práctica DRL - Tesis Portafolio

13 de abril de 2026

Introducción a la Fase Algorítmica

Habiendo dejado atrás el denso tejido matemático del Proceso de Decisión de Markov (Clase 2) y cementado las restricciones neuronales (Clase 3), nos adentramos en la arquitectura del software. En esta fase, trazamos el diseño del sistema físico a nivel fundacional antes de escribir una sola línea de código en Python.

Alineándonos estrictamente con las etapas dictadas en el documento "Memoria DRL", definiremos cómo interconectaremos la ingesta de datos, la simulación física de la bolsa y la auditoría final de viabilidad de la Inteligencia Artificial frente al modelo Lineal de GAMS.

1. 1. Ingesta Estricta del Dataset (El Ancla de Viabilidad)

La primera directriz inquebrantable de programación será la validación científica mediante control de variables (Etapas 1 y 2). Para garantizar una comparativa legítima entre la Red Neuronal y la Optimización Matemática, ambas inteligencias deben procesar información milimétricamente exacta a través de los mismos horizontes de tiempo (t_1 a t_{163}).

1.1. 1.1 Radiografía y Naturaleza del Dataset

Antes de manipular archivos, es imperativo comprender qué "sentidos" variables perceptivas le estaremos otorgando a la Inteligencia Artificial en cada paso cronológico t . En base a la estructura de tus datos históricos (`ps.gms`), el Agente DRL recibirá un vector numérico conformado por los siguientes parámetros financieros:

1. **Retornos Netos Puros ($R_{i,t}$):** Es la métrica cruda y definitiva de ganancia o pérdida porcentual de cada activo en la ventana de tiempo evaluada. Es el juez absoluto del mercado.
 - *Ejemplo Visual de Data:* En el periodo $t_1 \rightarrow R_{SPX} = 0,012$ (crecimiento del 1,2 %) y $R_{Cripto} = -0,05$ (caída del 5 %).
2. **Probabilidades del Régimen HMM ($P(bear_t)$, $P(bull_t)$):** El indicador cerebral de crisis. Muestra la métrica de creencia de en qué rastro oculto se encuentra el mercado actualmente.
 - *Ejemplo Visual de Data:* En un instante t_5 de alta volatilidad, el modelo dictamina: $P(bear) = 0,676$ y $P(bull) = 0,323$. La IA palpa este 67 % de crisis latente e intuitivamente aprende en base a castigos previos a refugiarse oportunamente.
3. **Vector de Medias Esperadas (μ_{mix}):** La proyección matemática de ganancia condicionada transversalmente por el Régimen HMM.

- *Ejemplo Visual de Data:* Para el paso temporal $t_5 \rightarrow \text{mumix}_{SPX} = 0,008$ y $\text{mumix}_{Cripto} = -0,015$.
4. **Matriz de Covarianza Mixta ($\text{sigma_mix}_{i,j,t}$):** El mapa de riesgo bidimensional (2×2) entre SPX y Cripto para ese periodo exacto. Le enseña a la IA en un solo pantallazo matricial cómo se correlacionan los activos bajo el régimen actual.
- *Ejemplo Visual de Data:* Una matriz diagonal donde observa el nivel de varianza riesgosa individual y en las esquinas opuestas capta si SPX sube cuando Cripto necesariamente baja.
5. **Pesos Históricos Heredados (w_{t-1}):** La propia "memoria remanente del agente". Para poder calcular el costo contable real de moverse hoy, el entorno le recuerda a la IA qué decisiones tomó ayer.
- *Ejemplo Visual de Data:* $w_{t-1} = [0,60, 0,40]$ (60 % SPX, 40 % Cripto). Si la IA hoy decide violentamente rebalancear a $[0,0, 1,0]$, el entorno le restará el 60 % de diferencia facturándole un porcentaje de comisión destructiva c_i .

Definición Formal del Vector de Estado (S_t generalizado): Todos estos elementos se concatenan y se "aplanan" (*flatten*) en un único vector unidimensional que sirve como entrada literal (*Input Layer*) a las redes Densas de nuestro Actor y Crítico. Matemáticamente, el estado matricial que absorbe la IA en cada tic temporal es:

$$S_t = \left[R_{SPX,t}, R_{Cripto,t}, P(bear_t), \mu_{SPX,t}, \mu_{Cripto,t}, vec(\Sigma_t), w_{SPX,t-1}, w_{Cripto,t-1} \right] \quad (1)$$

Nota Técnica: $P(bull_t)$ se puede omitir por colinealidad ya que $P(bull_t) = 1 - P(bear_t)$. Este vector numérico denso es el ojo con el que el algoritmo "ve" el tablero bursátil antes de jugar su turno impulsando los pesos de inversión a_t .

1.2. 1.2 Operativa Ciega de Carga (Data Handler)

Rechazaremos de tajo el *web scraping* en vivo (como usar librerías de Python para bajar precios crudos de Yahoo Finance de la web). En su lugar, el pipeline programático será instruido paramétricamente para extraer, limpiar y pivotar la data ancla contenida rígidamente en tus fuentes nativas de GAMS (aludiendo al archivo maestro `gdx.xlsx` o el código `ps.gms`):

- **Carga de la Condición de Inicio Exógena (Datos Base):** El código leerá matemáticamente los parámetros brutos de entrada que construyen el escenario financiero histórico base (probabilidades *prob_bear/bull* destiladas por el modelo HMM subyacente, tensores de media esperada *mumix* y covarianzas condicionales *sigma_mix*).
- **Aislamiento de Resultados:** Es crucial formalizar que **absolutamente nada de lo calculado u optimizado internamente por GAMS será ingerido por nuestra IA**. GAMS opera estricta y puramente como un Benchmark (Nuestro meta-competidor). Lo único que tomaremos de sus repositorios de datos (`gdx.xlsx` o `ps.gms`) son las constantes fundacionales inyectadas en su momento. Garantizamos así que la Red Neuronal comience la pelea en t_1 bajo la misma Condición de Inicio incorrupta.
- **Verdad Absoluta:** Los retornos netos ($R_{i,t}$) dictarán estrictamente si la IA gana o pierde a modo de *Ground Truth*.

Esto garantiza que la IA no tendrá excusas ni ventajas técnicas frente a Markowitz clásico.

2. Anatomía de la Simulación: El Entorno Customizado en Gymnasium

Gymnasium será nuestro patio de juegos, actuando como la "Física Bursátil" (Etapa 3 de la metodología). Diseñaremos nuestro entorno a imagen y semejanza del marco formal `PortfolioEnv`. Su flujo interno computacional se dividirá en dos motores primordiales dictando el ciclo de vida de los episodios:

2.1. Motor de Arranque (Función `reset(...)`)

Toda simulación debe comenzar en alguna parte.

- Obligará a la simulación a retornar a un paso inicial $t = 0$ o una ventana designada.
- Sembrará el Capital Inicial contable predeterminado (x_0).
- Inyectará el portafolio inerte base obligando a fijar vectores equitativos w_0 (ej. 50/50) para que la red neuronal y GAMS posean punto de arranque común en sus costos de transacciones al primer día.

2.2. Motor de Evolución Temporal (Función `step(...)`)

El núcleo algorítmico más destructivo y valioso. Tras recibir del Actor Neurológico PPO o SAC la orden bruta de inversión a_t :

1. Pasa los pesos sugeridos por un filtro probabilístico forzándolos a un hiperplano simplex ($w_{SPX} + w_{Cripto} = 1$) mediante el transformador `máx(0)`.
2. Extrae los números de rentabilidad real que dictó el mercado del dataset base para el tiempo t .
3. Computa el golpe financiero absoluto cobrando al broker los costos de transacción fijos: $c_i |w_t - w_{t-1}|$.
4. Actualiza el capital acumulado al cierre del periodo x_{t+1} .
5. **Emisión de Juicio Analítico:** Empaqueta los resultados y calcula *La Recompensa* iterativa r_t inyectando el riesgo, retorno y penalización. Se la estampa al Agente para que active Backpropagation y ajusta $w_{t-1} \leftarrow w_t$ preparándose para recibir el siguiente tic del horizonte.

3. Protocolo de Entrenamiento y Backtesting: Horizontes Rulantes (Walk-Forward)

Dado que el tiempo financiero muta violentamente, tu documento rechaza de lleno un particionado ingenuo clásico de 80 % Entrenamiento $\rightarrow$ 20 % Testeo (K-Fold normal). Entrar a ciegas al test violaría la hipótesis no estacionaria (No i.i.d.) de los mercados.

Técnica Walk-Forward Iterativa (Etapa 5): La inteligencia ejecutará ciclos expansivos temporales.

1. Entrenamos a la red desde t_1 hasta t_k .
2. Cortamos el aprendizaje y abrimos la cortina de Evaluación a futuro sobre t_{k+1} probando fuera de muestra (predicción fría).

3. La IA suma los datos de t_{k+1} al paquete de aprendizaje, se actualizan sus gradientes dinámicamente y con su nuevo nivel intelectual optimizado repite el reto prediciendo rigurosamente para el periodo estocástico de t_{k+2} .

Este bucle se estira cronológicamente y prohíbe terminantemente que la máquina sufra "Look-ahead Bias" (sesgo de mirar el futuro encubierto), consolidando la validación del portafolio en pura tracción temporal real evaluada paso a paso.

4. Tribunal de Métricas Analíticas y Benchmarking (Etapa 6)

El tramo terminal de la arquitectura involucra agrupar los resultados para rendir el juicio técnico de la matriz comparativa final: **PPO vs. SAC vs. Markowitz Lineal de GAMS**.

Una vez consumidos los datos del **walk-forward**, se trazarán gráficos de rentabilidad compuesta sobre ventanas y se destilará sistemáticamente el accionar de la IA utilizando ratios financieros convencionales que un tribunal evaluador clásico entendería:

- **Crecimiento de Capital Acumulado (Cumulative Return):** Quien terminó con las bolsillos más llenos descontando fricción perniciosa.
- **Máxima Caída Progresiva (Max Drawdown $\rightarrow$ MDD):** Evaluando con lupa la resiliencia en periodos bear-HMM de cada modelo comparativo antes de lograr un nuevo pico histórico.
- **Ratio de Sharpe Estocástico:** Ponderando la tasa pasiva libre de riesgo ≈ 0 sobre la desviación estándar empírica demostrada al correr del simulación en episodios.

Con este plano operacional riguroso finamente detallado conforme al manual de implementación de tu Tesis, nuestro sistema ya no posee vacíos arquitectónicos o procedimentales. Ahora sí, tenemos licencia intelectual aprobada para sumergirnos directamente en el ecosistema informático armando las tuberías en PyTorch y Stable-Baselines 3.

5. 5. Cronología Maestra del Proyecto DRL (Secuencia de Ejecución)

Para condensar toda tu visión organizativa, a continuación se detalla cronológicamente el camino paso a paso que recorrerá el dato desde su origen histórico hasta coronarse como el resultado final al terminar la Evaluación.

1. **Paso 1: Mapeo de la Base de Datos Histórica (El Origen):** El script en Python intercepta los archivos originales `.gdx` o `.gms` y extrae los retornos R_t , las matrices de covarianza condicionales Σ , los tensores *mumix* y las probabilidades rígidas entregadas por el algoritmo HMM subyacente.
2. **Paso 2: Preprocesamiento (Data Handler):** Se ordenan todos los tensores cronológicamente, garantizando que el tiempo coincida (ej. filtrando para empezar desde 2014) y se dejan listos para ser consumidos por paso de simulación.
3. **Paso 3: Construcción de la Realidad Bursátil (t):** En el instante t , el entorno *Gymnasium* empaqueta el estado del mercado más la memoria del peso accionario del agente del día anterior (w_{t-1}). Todo se ensambla y aplana originando el Vector de Observación S_t .
4. **Paso 4: Predicción Neurológica (Inferencia de la Política π):** El Actor de la Red (Cerebro SAC/PPO) recibe el vector S_t .
 - **Actor** $\rightarrow \pi_\theta$: La red no escupe el peso w directamente, sino los parámetros (μ, σ) de una distribución de probabilidad $\pi_\theta(a_t|S_t)$.
 - **Muestreo (Sampling):** Se extrae una acción estocástica de esa distribución, la cual representa los pesos financieros crudos a_t .
 - **Activación Final:** Estos pesos pasan por una capa Softmax o transformación de suma-1 para convertirse en el portafolio ejecutable w_t .
5. **Paso 5: Trance y Evolución del Entorno:** La acción se inyecta en Gymnasium.
 - Se aplican recortes estomacales cobrando tasas de coste de transacción por rotar el portafolio.
 - Se pondera el nuevo vector w_t por el retorno real del mercado en t .
 - Se calcula la métrica unificada de valor (La Recompensa r_t).
6. **Paso 6: El Duelo de Aprendizaje (PPO vs. SAC) y Actualización de Redes:** En esta fase, el software minimiza funciones de "Pérdida"(Loss) mediante el algoritmo de retropropagación. Es fundamental entender que aunque el objetivo es ganar dinero, el optimizador (ADAM) siempre busca el mínimo de una función de error.
 - **Pérdida del Actor (Actor Loss):**
 - **Vía PPO (Clip Loss):** Minimiza $L_{actor} = -\hat{\mathbb{E}}_t[\text{mín}(r_t(\theta)\hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon)\hat{A}_t)]$. El signo negativo convierte el objetivo de maximizar ganancia en una minimización de error.
 - **Vía SAC (Entropy Loss):** Minimiza $L_{actor} = \mathbb{E}_t[\alpha \log \pi_\theta(a_t|S_t) - Q_\phi(S_t, a_t)]$. Al minimizar esta suma, obligamos a la red a que Q crezca y que la probabilidad de acciones diversas (entropía) se mantenga alta.
 - **Pérdida del Crítico (Critic Loss):** Se utiliza el **Error Cuadrático Medio (MSE)** sobre el error de Bellman:

$$L_{critic} = \mathbb{E}_t \left[(Q_{pred} - (r_t + \gamma Q_{target}))^2 \right] \quad (2)$$

- **Optimización Adaptativa (ADAM):** En lugar de una tasa de aprendizaje fija, usamos el algoritmo **ADAM**. Este ajusta los pesos θ usando el primer momento m_t (media) y el segundo momento v_t (varianza no centrada) de los gradientes:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} L, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} L)^2 \quad (3)$$

La actualización final de cada peso es: $\theta_{new} \leftarrow \theta_{old} - \frac{\eta}{\sqrt{v_t} + \epsilon} \hat{m}_t$, donde η se adapta dinámicamente según el ruido bursátil captado en los momentos previos.

- Paso 7: Horizonte Rodante (Validación Walk-Forward):** Una vez entrenada óptimamente la red para el set de años, avanza la cortina del tiempo para $t + 1$ (ver el mercado desconocido del siguiente mes). Se guarda el rendimiento de esa predicción ciega acumulando un historial de éxito fuera de la muestra.
- Paso 8: Sentencia Final (El Benchmark):** Terminado el bucle a lo largo de todos los periodos, se apilan las rentabilidades de nuestra Inteligencia Artificial vs. las optimizaciones lineales estáticas entregadas previamente por GAMS. Se trazan tablas cruzadas de Sharp Ratio y Max Drawdowns, arrojando un ganador empírico sobre papel.

6. Laboratorio Numérico: De la Data al Peso (Desarrollo Total)

Para visualizar la "física interna" que ocurrirá en tu código Python, desarrollaremos matemáticamente las primeras dos iteraciones (t_1, t_2) de un Agente PPO simplificado (1 capa oculta de 2 neuronas) gestionando 2 activos ($SPX, Cripto$).

6.1. Configuración Inicial del Sistema

- **Cerebro (Actor θ):** Pesos iniciales $W_1 = \begin{bmatrix} 0,1 & 0,1 \\ 0,1 & 0,1 \end{bmatrix}$ (simplificación), Sesgos $b = 0$.
- **Parámetros Financieros:** Costo transaccional $c = 0,01$ (1%), Riesgo $\lambda = 2$, Capital $x_0 = 100$.
- **Data Maestro (t_1):** $P(bear) = 0,1$. $\mu_{mix} = \begin{bmatrix} 0,008 \\ 0,022 \end{bmatrix}$, $\Sigma_{mix} = \begin{bmatrix} 0,0001 & 0 \\ 0 & 0,001 \end{bmatrix}$. Portafolio previo $w_{t-1} = \begin{bmatrix} 0,5 \\ 0,5 \end{bmatrix}$.

6.2. Iteración 1 (t_1): El Primer Paso

Paso A: Ingesta y Vector de Estado (S_1) Concatenamos las señales filtrando redundancias: $S_1 = [\mu_{SPX}, \mu_{Cripto}, P(bear), w_{SPX,0}, w_{Cripto,0}] = [0,008, 0,022, 0,1, 0,5, 0,5]$

Paso B: Feed-Forward (Cerebro IA) 1. Multiplicación Lineal: $z_1 = S_1 \cdot W_1 = [0,008 + 0,022 + 0,1 + 0,5 + 0,5] \cdot 0,1 = [0,113, 0,113]$. 2. Activación ReLU: $h_1 = \max(0, z_1) = [0,113, 0,113]$. 3. Salida y Simplex: Como los valores son iguales, la proyección dictamina la acción: $w_{SPX,1} = 0,5$ y $w_{Cripto,1} = 0,5$. La IA decide no moverse aún.

Paso C: Ejecución en Gymnasium ($t_1 \rightarrow t_2$) El mercado responde con retornos reales: $R_1 = [0,01, -0,02]$. 1. Retorno Portafolio: $0,5(0,01) + 0,5(-0,02) = -0,005$. 2. Costos de Transacción: $0,01 \cdot (|0,5 - 0,5| + |0,5 - 0,5|) = 0$. 3. Capital Final: $x_1 = 100 \cdot (1 - 0,005) = 99,5$. 4. **Cálculo de Recompensa (r_1):** $r_1 = w^\top \mu_{mix} - \lambda(w^\top \Sigma_{mix} w) - \text{Costo}$ $r_1 = 0,015 - 2(0,25 \cdot 0,0001 + 0,25 \cdot 0,001) - 0 = 0,01445$.

Paso D: Backpropagation y ADAM (El Aprendizaje) El Crítico evalúa el estado actual: $V(S_1) = 0,01$ y el estado siguiente: $V(S_2) = 0,012$. PPO no posee red Q ; la aproxima

vía Bellman: $Q \approx r_t + \gamma V(S_{t+1})$. Con $\gamma = 0,99$: $Q_1 \approx 0,01445 + 0,99 \cdot 0,012 = 0,02633$ $A_1 = Q_1 - V(S_1) = 0,02633 - 0,01 = 0,01633$ (ventaja positiva $\rightarrow$ buena decisión). 1. **Gradiente de la Loss (L_{actor})**: Para un peso w_{ij} de la red: $\frac{\partial L}{\partial w_{ij}} = -A_1 \cdot \frac{\partial \log \pi}{\partial w_{ij}}$. Asumamos que calculamos $\nabla_{\theta} L = -0,002$ para el peso $W_{1,1}$. 2. **Actualización ADAM (Simplificada)** $\beta_1 = 0,9, \beta_2 = 0,99, \eta = 0,01$: $m_1 = 0,9(0) + 0,1(-0,002) = -0,0002$ $v_1 = 0,99(0) + 0,01(-0,002)^2 \approx 0$ $W_{new} = 0,1 - (0,01 \cdot \frac{-0,0002}{\sqrt{1e-8}}) \approx 0,1002$ (El peso sube un poco).

6.3. Iteración 2 ($t = 2$): La Evolución

Ahora la red tiene pesos actualizados θ_2 y el dataset entrega nueva información: **Data Maestro (t_2)**: $P(bear) = 0,914$ (Pánico). $w_{t-1} = [0,5, 0,5]$. 1. **Nuevo Estado S_2** : El componente de pánico 0,914 ahora es masivo en el vector. 2. **Reacción Neuronal**: Gracias al ajuste de pesos previo y al nuevo input, el Actor dispara una salida $\tilde{a}_2 = [3,5, 0,1]$. 3. **Nuevo Portafolio w_2** : $\frac{3,5}{3,6} \approx 0,97$ y $\frac{0,1}{3,6} \approx 0,03$. 4. **Resultado**: La IA ha aprendido a huir de las Criptos en t_2 ante la señal del HMM. Se calculan costos $c \cdot |0,97 - 0,5| = 0,0047$, mermando el capital pero salvando el grueso de la caída bursátil.